

Learn Django By Creating the TODO App

Part 1 : Project Setup

Step 1: Setting Up the Project

1. Install Python and Django

Make sure you have **Python 3.8+** installed. Then, in your terminal:

```
# Create a virtual environment
python -m venv venv

# Activate it
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# Install Django
pip install django
```

Check installation:

```
django-admin --version
```

2. Create a Django Project

Let's create our main project called `todo_project`:

```
django-admin startproject todo_project
cd todo_project
```

This creates a structure like:

```
todo_project/
  manage.py
  todo_project/
    __init__.py
    settings.py
    urls.py
    asgi.py
    wsgi.py
```

3. Run the Development Server

Check if Django runs correctly:

```
python manage.py runserver
```

Then go to <http://127.0.0.1:8000>. If you see the **Django rocket page**, everything's working 🚀.

Step 2: Create the To-Do App

Inside the project directory, create an app:

```
python manage.py startapp todo
```

Now you have:

```
todo/  
  migrations/  
  __init__.py  
  admin.py  
  apps.py  
  models.py  
  tests.py  
  views.py
```

Then, register this app in `todo_project/settings.py`:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'todo', # our new app  
]
```

Step 3: Create the To-Do Model

Open `todo/models.py` and define a simple model:

```
from django.db import models  
  
class Task(models.Model):  
    title = models.CharField(max_length=200)  
    completed = models.BooleanField(default=False)  
  
    def __str__(self):  
        return self.title
```

This defines a **Task** with:

- `title`: the name of the task
- `completed`: whether it's done

Now, apply migrations:

```
python manage.py makemigrations  
python manage.py migrate
```

Step 4: Register Model in the Admin Panel

Open `todo/admin.py`:

```
from django.contrib import admin  
from .models import Task  
  
admin.site.register(Task)
```

Create an admin user to test it:

```
python manage.py createsuperuser
```

Then run the server and go to `/admin` to log in.

At this point:

1. Django is set up
 2. App created
 3. Model and admin ready
-

Part 2: Building the Views and Templates (UI)!

Now we'll make our app interactive — users will be able to:

- View all tasks
- Add a new task
- Delete a task
- Mark a task as completed

We'll also use **Bootstrap** for a clean UI.

Step 5: Set Up URLs and Views

1. Create a URL file for the `todo` app

Inside your `todo/` folder, create a new file called `urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('add/', views.add_task, name='add_task'),
    path('delete/<int:task_id>/', views.delete_task, name='delete_task'),
    path('complete/<int:task_id>/', views.complete_task, name='complete_task'),
]
```

Then, include this file in the **main project URL** configuration.

Open `todo_project/urls.py` and modify it like this:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('todo.urls')), # include our app's URLs
]
```

Step 6: Create Views

Open `todo/views.py` and add the following code:

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Task

# Show all tasks
def index(request):
    tasks = Task.objects.all().order_by('-id')
    return render(request, 'todo/index.html', {'tasks': tasks})

# Add a new task
def add_task(request):
    if request.method == 'POST':
        title = request.POST.get('title')
        if title:
            Task.objects.create(title=title)
    return redirect('index')

# Delete a task
def delete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id)
    task.delete()
    return redirect('index')

# Mark a task as completed
def complete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id)
    task.completed = True
    task.save()
    return redirect('index')
```

Step 7: Create Templates (UI)

We'll use **Bootstrap 5** for styling.

1. Create a templates folder

Inside your `todo` app, create:

```
todo/
├── templates/
│   └── todo/
│       └── index.html
```

2. Add HTML for the main page (`index.html`)

Paste this code inside `index.html` :

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do App</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container mt-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h2 class="text-center mb-4"> My To-Do List</h2>

      <!-- Add Task Form -->
      <form action="{% url 'add_task' %}" method="POST" class="d-flex mb-3">
        {% csrf_token %}
        <input type="text" name="title" class="form-control me-2" placeholder="Enter new task..." required>
        <button type="submit" class="btn btn-primary">Add</button>
      </form>

      <!-- Task List -->
      <ul class="list-group">
        {% for task in tasks %}
          <li class="list-group-item d-flex justify-content-between align-items-center">
            <div>
              {% if task.completed %}
                <span class="text-success text-decoration-line-through">{{ task.title }}</span>
              {% else %}
                {{ task.title }}
              {% endif %}
            </div>
            <div>
              {% if not task.completed %}
                <a href="{% url 'complete_task' task.id %}" class="btn btn-sm btn-success">Done</a>
              {% endif %}
              <a href="{% url 'delete_task' task.id %}" class="btn btn-sm btn-danger">Delete</a>
            </div>
          </li>
          {% empty %}
            <li class="list-group-item text-center text-muted">No tasks yet!</li>
          {% endfor %}
        </ul>
      </div>
    </div>
  </div>

</body>
</html>

```

Step 8: Test It!

Now run your development server again:

```
python manage.py runserver
```

Go to <http://127.0.0.1:8000>

You should now see:

- A Bootstrap UI
- Ability to **add**, **mark as done**, and **delete** tasks

Congratulations! You've built a **fully working To-Do app** with Django + UI.

Next Steps

Now, we can extend this app with:

1. **Edit tasks**
 2. **User authentication** (login/logout)
 3. **Dark/light theme toggle**
 4. **Responsive front-end with Django REST + JS**
-

Part 3 - Update forms, pass data to templates, and manage POST requests in Django

🔪 Step 9: Add “Edit Task” Feature

We'll do this in **4 steps**:

1. Add a new URL for editing
 2. Create a view to handle GET + POST requests
 3. Update the HTML template
 4. Test it in the browser
-

1 🔪 Add URL for Editing

Open **todo/urls.py** and add this line:

```
path('edit/<int:task_id>/', views.edit_task, name='edit_task'),
```

Your file should now look like:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('add/', views.add_task, name='add_task'),
    path('delete/<int:task_id>/', views.delete_task, name='delete_task'),
    path('complete/<int:task_id>/', views.complete_task, name='complete_task'),
    path('edit/<int:task_id>/', views.edit_task, name='edit_task'), # 🔪 new
]
```

2 🔪 Add the View in **views.py**

Open **todo/views.py** and add this new function:

```
def edit_task(request, task_id):
    task = get_object_or_404(Task, id=task_id)

    if request.method == 'POST':
        new_title = request.POST.get('title')
        if new_title:
            task.title = new_title
            task.save()
            return redirect('index')

    return render(request, 'todo/edit.html', {'task': task})
```

Explanation:

- If the user opens `/edit/1/` → it shows a form with the current title.
- When they submit it (`POST`), we update the task and redirect back to the main list.

3🔗 Create the Edit Template

Inside your `todo/templates/todo/` folder, create a new file: `edit.html`

Paste this:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Edit Task</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">

<div class="container mt-5">
  <div class="card shadow-sm">
    <div class="card-body">
      <h3 class="text-center mb-4">🔗 Edit Task</h3>

      <form method="POST">
        {% csrf_token %}
        <div class="mb-3">
          <input type="text" name="title" value="{{ task.title }}" class="form-control" required>
        </div>
        <button type="submit" class="btn btn-success w-100 mb-2">Save Changes</button>
        <a href="{% url 'index' %}" class="btn btn-secondary w-100">Cancel</a>
      </form>
    </div>
  </div>
</div>

</body>
</html>
```

4🔗 Add an “Edit” Button to the Main Page

Open `index.html` again, and in the task buttons section, add the edit link:

```
<div>
  {% if not task.completed %}
    <a href="{% url 'complete_task' task.id %}" class="btn btn-sm btn-success">Done</a>
    <a href="{% url 'edit_task' task.id %}" class="btn btn-sm btn-warning">Edit</a> <!-- 🆕 new -->
  {% endif %}
  <a href="{% url 'delete_task' task.id %}" class="btn btn-sm btn-danger">Delete</a>
</div>
```

Now your tasks have a “🔗 Edit” button.

5🔗 Test It!

Run the server again:

```
python manage.py runserver
```

Go to <http://127.0.0.1:8000>

- Click **Edit** next to any task
- Change its text
- Hit “Save Changes”

The page will return to your task list with the updated title!

Note

You’ve now learned:

- How to use dynamic URLs (<int:id>)
- How to send an existing object to a template
- How to handle POST form updates in Django

Part 4 - user authentication

🔒 Step 10: Add User Authentication

We’ll implement:

- 🔒 **User registration**
- 🔒 **Login**
- 🔒 **Logout**
- 🔒 **Personal tasks per user**

1 🔒 Update the Task Model

We need to associate each task with a user. Open `todo/models.py` and modify the `Task` model:

```
from django.db import models
from django.contrib.auth.models import User

class Task(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE) # 🔒 new
    title = models.CharField(max_length=200)
    completed = models.BooleanField(default=False)

    def __str__(self):
        return self.title
```

Now, each task belongs to a **User**.

Run migrations:

```
python manage.py makemigrations
python manage.py migrate
```

2 🔒 Update Views to Filter Tasks by User

In `views.py`, update the `index` view:

```
from django.contrib.auth.decorators import login_required

@login_required
def index(request):
    tasks = Task.objects.filter(user=request.user).order_by('-id')
    return render(request, 'todo/index.html', {'tasks': tasks})
```

Also, in `add_task`:


```
@login_required
def add_task(request):
    if request.method == 'POST':
        title = request.POST.get('title')
        if title:
            Task.objects.create(user=request.user, title=title)
    return redirect('index')
```

Do the same for **delete_task**, **complete_task**, and **edit_task**:

```
@login_required
def delete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id, user=request.user)
    task.delete()
    return redirect('index')

@login_required
def complete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id, user=request.user)
    task.completed = True
    task.save()
    return redirect('index')

@login_required
def edit_task(request, task_id):
    task = get_object_or_404(Task, id=task_id, user=request.user)
    if request.method == 'POST':
        new_title = request.POST.get('title')
        if new_title:
            task.title = new_title
            task.save()
    return redirect('index')
    return render(request, 'todo/edit.html', {'task': task})
```

Using `user=request.user` ensures users can only modify their own tasks.

3 Create URLs for Authentication

In `todo/urls.py`, add:

```
from django.contrib.auth import views as auth_views
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('add/', views.add_task, name='add_task'),
    path('delete/<int:task_id>/', views.delete_task, name='delete_task'),
    path('complete/<int:task_id>/', views.complete_task, name='complete_task'),
    path('edit/<int:task_id>/', views.edit_task, name='edit_task'),

    # Auth URLs
    path('login/', auth_views.LoginView.as_view(template_name='todo/login.html'), name='login'),
    path('logout/', auth_views.LogoutView.as_view(next_page='login'), name='logout'),
    path('register/', views.register, name='register'),
]
```

4 Create Registration View

In `views.py`, add:

```

from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import login
from django.shortcuts import render, redirect

def register(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user) # Automatically log in new user
            return redirect('index')
    else:
        form = UserCreationForm()
    return render(request, 'todo/register.html', {'form': form})

```

5 Create Templates

login.html

```

{% extends 'todo/base.html' %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h3 class="text-center mb-4"> Login</h3>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit" class="btn btn-primary w-100">Login</button>
    </form>
    <p class="text-center mt-2">Don't have an account? <a href="{% url 'register' %}">Register</a></p>
</div>
{% endblock %}

```

register.html

```

{% extends 'todo/base.html' %}
{% block content %}
<div class="container mt-5" style="max-width: 400px;">
    <h3 class="text-center mb-4"> Register</h3>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit" class="btn btn-success w-100">Register</button>
    </form>
    <p class="text-center mt-2">Already have an account? <a href="{% url 'login' %}">Login</a></p>
</div>
{% endblock %}

```

base.html

We'll create a base template for consistent styling:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>To-Do App</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
    {% if user.is_authenticated %}
    <nav class="navbar navbar-expand-lg navbar-light bg-white shadow-sm mb-4">
        <div class="container">
            <a class="navbar-brand" href="{% url 'index' %}"> To-Do</a>
            <div>
                <span class="me-3">Hello, {{ user.username }}!</span>
                <a class="btn btn-outline-danger btn-sm" href="{% url 'logout' %}">Logout</a>
            </div>
        </div>
    </nav>
    {% endif %}
    {% block content %}{% endblock %}
</body>
</html>

```

6 Redirect to Login if Not Authenticated

In `settings.py`, add:

```

LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'index'
LOGOUT_REDIRECT_URL = 'login'

```

This ensures users are **forced to log in** before accessing tasks.

7 Test the Authentication System

1. Go to `/register/` → create a new user
2. Log in → you'll see your personal task list
3. Add tasks → only visible to your user
4. Logout → redirected to login page

Now your app is **multi-user ready**!

Part 5 - Dynamic with AJAX so users can add, complete, or delete tasks without refreshing the page.

We'll use **jQuery** (simplest for Django + AJAX) for this .

Step 11: Add AJAX for Dynamic UI

We'll focus on three actions:

1. Add task
2. Complete task
3. Delete task

1 Update Templates for AJAX

Open `index.html` and make these changes:

a) Include jQuery

Add this inside `<head>` :

```
<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
```

b) Update the Add Task Form

Change the form to **prevent default submission**:

```
<form id="add-task-form" class="d-flex mb-3">
  {% csrf_token %}
  <input type="text" name="title" id="task-title" class="form-control me-2" placeholder="Enter new task..." required>
  <button type="submit" class="btn btn-primary">Add</button>
</form>
```

c) Wrap Task List for AJAX Updates

```
<ul class="list-group" id="task-list">
  {% for task in tasks %}
    <li class="list-group-item d-flex justify-content-between align-items-center" data-id="{{ task.id }}">
      <div>
        {% if task.completed %}
          <span class="text-success text-decoration-line-through">{{ task.title }}</span>
        {% else %}
          {{ task.title }}
        {% endif %}
      </div>
      <div>
        {% if not task.completed %}
          <button class="btn btn-sm btn-success complete-btn">Done</button>
          <a href="{% url 'edit_task' task.id %}" class="btn btn-sm btn-warning">Edit</a>
        {% endif %}
          <button class="btn btn-sm btn-danger delete-btn">Delete</button>
        </div>
    </li>
  {% empty %}
    <li class="list-group-item text-center text-muted">No tasks yet!</li>
  {% endfor %}
</ul>
```

2❏ Update Django Views for AJAX

In `views.py`, we'll check if the request is AJAX and return **JSON**:

```

from django.http import JsonResponse

@login_required
def add_task(request):
    if request.method == 'POST' and request.headers.get('x-requested-with') == 'XMLHttpRequest':
        title = request.POST.get('title')
        if title:
            task = Task.objects.create(user=request.user, title=title)
            return JsonResponse({
                'id': task.id,
                'title': task.title,
                'completed': task.completed
            })
    return redirect('index')

@login_required
def complete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id, user=request.user)
    task.completed = True
    task.save()
    if request.headers.get('x-requested-with') == 'XMLHttpRequest':
        return JsonResponse({'status': 'success', 'task_id': task.id})
    return redirect('index')

@login_required
def delete_task(request, task_id):
    task = get_object_or_404(Task, id=task_id, user=request.user)
    task.delete()
    if request.headers.get('x-requested-with') == 'XMLHttpRequest':
        return JsonResponse({'status': 'success', 'task_id': task_id})
    return redirect('index')

```

3 Add AJAX Script in index.html

Place this at the bottom before `</body>`:

```

<script>
$(document).ready(function() {
    // CSRF setup for AJAX
    function getCookie(name) {
        let cookieValue = null;
        if (document.cookie && document.cookie !== '') {
            const cookies = document.cookie.split(';');
            for (let i = 0; i < cookies.length; i++) {
                const cookie = cookies[i].trim();
                if (cookie.substring(0, name.length + 1) === (name + '=')) {
                    cookieValue = decodeURIComponent(cookie.substring(name.length + 1));
                    break;
                }
            }
        }
    }
    return cookieValue;
}

const csrftoken = getCookie('csrftoken');

// Add Task
$('#add-task-form').submit(function(e) {
    e.preventDefault();
    const title = $('#task-title').val();
    $.ajax({
        url: "{% url 'add_task' %}",

```

```

    type: 'POST',
    data: { 'title': title },
    headers: { 'X-CSRFToken': csrftoken, 'X-Requested-With': 'XMLHttpRequest' },
    success: function(data) {
        const newTask = `<li class="list-group-item d-flex justify-content-between align-items-center" data-id="${data.id}">
            <div>${data.title}</div>
            <div>
                <button class="btn btn-sm btn-success complete-btn">Done</button>
                <a href="/edit/${data.id}/" class="btn btn-sm btn-warning">Edit</a>
                <button class="btn btn-sm btn-danger delete-btn">Delete</button>
            </div>
        </li>`;
        $('#task-list').prepend(newTask);
        $('#task-title').val('');
    }
});

// Complete Task
$('#task-list').on('click', '.complete-btn', function() {
    const li = $(this).closest('li');
    const taskId = li.data('id');
    $.ajax({
        url: `/complete/${taskId}/`,
        type: 'POST',
        headers: { 'X-CSRFToken': csrftoken, 'X-Requested-With': 'XMLHttpRequest' },
        success: function() {
            li.find('div:first').html(`<span class="text-success text-decoration-line-through">${li.find('div:first').text()}</span>`);
            li.find('.complete-btn').remove();
        }
    });
});

// Delete Task
$('#task-list').on('click', '.delete-btn', function() {
    const li = $(this).closest('li');
    const taskId = li.data('id');
    $.ajax({
        url: `/delete/${taskId}/`,
        type: 'POST',
        headers: { 'X-CSRFToken': csrftoken, 'X-Requested-With': 'XMLHttpRequest' },
        success: function() {
            li.remove();
        }
    });
});
});
</script>

```

4 Test Your Dynamic To-Do App

- Go to /
- Add tasks → appears instantly
- Mark tasks as done → updates instantly
- Delete tasks → disappears instantly

No page refresh needed! 🚀

Note :

You've now built a **full-stack dynamic To-Do app with Django**:

- Models, Views, Templates
- CRUD operations

- User Authentication
- AJAX for dynamic front-end

This is a **real-world Django application structure**.

Part 6- Deployment

Perfect! 🎉 Let's go step by step to **deploy your Django To-Do app** online so anyone can access it. I'll focus on **deployment to Render**, which is beginner-friendly, free-tier, and works well with Django. You can also deploy to **Heroku**, **PythonAnywhere**, or **VPS**, but Render is simpler for modern Django apps.

We'll cover:

1. Preparing your project
2. Setting up static files
3. Configuring for production
4. Creating a Render account
5. Connecting your project to Render
6. Migrating the database
7. Accessing your live app

🔪 Step 1: Prepare Your Project for Deployment

a) Install required packages

Make sure you have:

```
pip install gunicorn psycpg2-binary dj-database-url python-decouple whitenoise
```

- **gunicorn** → WSGI server
- **psycpg2-binary** → PostgreSQL adapter
- **dj-database-url** → Easy database config from URL
- **python-decouple** → Manage environment variables
- **whitenoise** → Serve static files in production

b) Create a `requirements.txt`

```
pip freeze > requirements.txt
```

This file tells Render what packages to install.

c) Create a `Procfile`

At the project root, create a file called **Procfile** :

```
web: gunicorn project_name.wsgi --log-file -
```

Replace `project_name` with your Django project folder (where `settings.py` lives).

d) Create `.env` for environment variables

In your root folder, create **.env** :

```
SECRET_KEY=your_django_secret_key
DEBUG=False
ALLOWED_HOSTS=your_app.onrender.com
DATABASE_URL=postgres://user:password@host:port/dbname
```

We'll use **Render's PostgreSQL** later.

Update `settings.py` to use **decouple**:

```
from decouple import config
import dj_database_url

SECRET_KEY = config('SECRET_KEY')
DEBUG = config('DEBUG', default=False, cast=bool)
ALLOWED_HOSTS = config('ALLOWED_HOSTS', cast=lambda v: [s.strip() for s in v.split(',')])

DATABASES = {
    'default': dj_database_url.config(default=config('DATABASE_URL'))
}

# Static files
STATIC_URL = '/static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
STATICFILES_STORAGE = 'whitenoise.storage.CompressedManifestStaticFilesStorage'
```

e) Collect static files

```
python manage.py collectstatic
```

📌 Step 2: Push Your Project to GitHub

Render needs your project in a Git repo.

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/yourusername/yourrepo.git
git push -u origin main
```

📌 Step 3: Create a Render Account and App

1. Go to [Render.com](https://render.com) → sign up
2. Click **New** → **Web Service**
3. Connect your **GitHub repo**
4. Configure:
 - **Environment:** Python
 - **Build Command:** `pip install -r requirements.txt && python manage.py collectstatic --noinput`
 - **Start Command:** `gunicorn project_name.wsgi`
 - **Environment Variables** → add SECRET_KEY, DEBUG, ALLOWED_HOSTS

Optional: Add PostgreSQL Database

1. Render → **New** → **Database** → **PostgreSQL**
2. Copy the **DATABASE_URL** to your `.env` or Render environment variable

Update `settings.py` if you haven't already (see Step 1d).

📌 Step 4: Migrate the Database

Once your app is live on Render:

1. Go to **Shell** in Render dashboard → connect to your service
2. Run migrations:


```
python manage.py migrate
python manage.py createsuperuser # Optional
```

🔑 Step 5: Access Your Live App

- After build succeeds, Render provides a URL like: `https://your-app.onrender.com`
 - Open it → your Django To-Do app is live!
-

Deployment Steps

1. Prepare Django for production (`DEBUG=False` , `ALLOWED_HOSTS` , static files, database config)
 2. Push to GitHub
 3. Create Render Web Service
 4. Configure environment variables
 5. Run migrations
 6. Access your live app
-

Tip: For smoother UX, enable **HTTPS** in Render (free) and make sure all static files load properly.
